
UrduStack

A Code-Switch-Aware Urdu NLP Infrastructure for Scam and Harassment Detection

Technical Report

Authors	Shahoud Shahid, Munaza Tariq
Repository	github.com/munazat/UrduStack
Date	September 2026
Deployment	Hugging Face Spaces / Google Colab (T4 GPU)

Abstract

Pakistani digital text routinely mixes Urdu script, Roman Urdu, and English within a single sentence, which lets scam and harassment content slip past moderation tools built for a single script. UrduStack is a self-hosted NLP pipeline that first normalizes code-switched text into Urdu script and then classifies it for scam and toxic content using a LoRA-fine-tuned XLM-RoBERTa model combined with a hardened heuristic layer in a max-ensemble. The system reaches 89.7% accuracy and an F1 score of 87.1% on a 107-example held-out test set, and passes 12 of 12 adversarial red-team cases against the deployed model — a result independently driven by the trained model on 8 of those 12 cases, not the keyword layer alone. On a separate probe of 11 phrases matching none of the heuristic's known patterns, the trained model correctly generalized on 7, demonstrating genuine semantic understanding rather than memorized keyword matching. The system additionally performs named-entity recognition, Urdu speech-to-text transcription, and plain-language explanation, and is reachable through a website, a Gradio demo, and a WhatsApp bot — all backed by the same scoring logic. This report documents the system's architecture, methodology, implementation, evaluation, and — deliberately, in equal detail — its known, currently open limitations.

Acknowledgements

We thank the organizers of this hackathon for the opportunity and the tight, motivating deadline. This project builds on open datasets and open-source software without which it would not exist: the Roman-Urdu-Parl parallel corpus and the Roman-Urdu-Toxic-Corpus (PURUTT) from the Hugging Face Hub community, the XLM-RoBERTa base model from Facebook AI, the PEFT/LoRA implementation from Hugging Face, OpenAI's Whisper speech-recognition model, and the FastAPI, Gradio, and PyTorch open-source projects.

Table of Contents

Abstract	2
Acknowledgements	3
Table of Contents	4
List of Figures	6
List of Tables	7
1. Introduction	8
1.1 Report Structure	8
2. Problem Statement	9
3. Motivation	10
4. Objectives	11
5. Background and Existing Approaches	12
5.1 English-Centric Moderation Tools	12
5.2 General-Purpose Large Language Models	12
5.3 Urdu-Specific NLP Models	12
6. Proposed Solution	13
7. System Architecture	14
8. Methodology	15
8.1 Model Fine-Tuning	15
8.2 Calibration	15
8.3 Evaluation Methodology	15
9. Detailed System Design	16
9.1 Normalization Cascade	16
9.2 Risk Scoring — the Max-Ensemble	16
9.3 Threat Categorization and Recommendation	16
10. Technologies and Tools	18

11. Implementation	19
11.1 Graceful Degradation	19
11.2 Active Learning Loop	19
12. Algorithms, Models, and Pipelines	20
13. API and Data Design	21
14. UI/UX Design	22
15. Features	23
16. Experimental Setup	24
17. Testing and Validation	25
17.1 Unit Tests	25
17.2 End-to-End Validation	25
17.3 Continuous Integration	25
18. Results	26
18.1 Risk Classifier Performance	26
18.2 Adversarial Red-Team Results	26
18.3 Novel-Vocabulary Generalization	26
18.4 Known, Currently Unfixed False Positives	27
18.5 UI/UX Verification	27
19. Limitations	30
20. Future Work	31
21. Conclusion	32
References	33
Appendix A: Full API Endpoint Reference	34
Appendix B: Repository Structure (abridged)	34

List of Figures

(See figures throughout the report; index below is generated from this build.)

List of Tables

(See tables throughout the report; index below is generated from this build.)

1. Introduction

UrduStack is a unified natural-language-processing pipeline purpose-built for the way people in Pakistan actually write online: a fluid mix of Urdu script, Roman Urdu (Urdu transliterated into Latin letters), and English, often within a single sentence. It was built as a two-day hackathon MVP and has since been iteratively hardened, tested against its own claims, and re-verified against a real trained model rather than left as an untested prototype.

The system's core contribution is not a single novel algorithm but a working, evaluated pipeline: normalization that survives code-switching, a risk classifier that combines a fine-tuned transformer with a fast heuristic layer, entity recognition, speech-to-text, and plain-language explanation — wired together behind one API and reachable from three different real-world surfaces.

1.1 Report Structure

Sections 2–5 establish the problem, its real-world stakes, and the project's objectives. Sections 6–10 describe the proposed solution, its architecture, methodology, and detailed design. Sections 11–15 cover implementation specifics: the models, the pipelines, the API surface, and the user-facing interfaces. Sections 16–18 present the experimental setup, testing methodology, and results — including cases where the system currently falls short, reported with the same rigor as its successes. Sections 19–21 close with limitations, future work, and conclusions.

2. Problem Statement

Existing content-moderation and scam-detection tools are largely built and evaluated on English or single-script text. Pakistani digital communication routinely violates that assumption. A single scam job posting observed during this project's development reads, verbatim:

```
"Urgent hiring! 50000 per week, send processing fee to register."
```

This sentence is entirely in Roman Urdu and English — no Urdu script appears at all — yet it carries a scam pattern (an advance-fee request tied to a job offer) that a Pakistani reader recognizes instantly. A naive English keyword filter for "processing fee" catches the unobfuscated case, but the same filter is defeated by trivial evasions: leetspeak ("pr0c3ss1ng f33"), character spacing ("p r o c e s s i n g f e e"), or simple misspellings ("procesing fee").

Two concrete harms motivate this project. First, fake job postings circulated on Facebook, WhatsApp, and OLX target Pakistani students and jobseekers, extracting advance "processing fees" for positions that do not exist. Second, harassing and abusive content in the same mixed script is similarly under-served by moderation tools tuned for a single language or script.

3. Motivation

Three considerations shaped this project's direction beyond the problem statement itself:

- **Explainability matters as much as accuracy.** A risk score with no explanation is not actionable for a non-technical user; the system needed to show which phrases drove a verdict and why, not just output a number.
- **Self-hosting is a real privacy requirement, not a preference.** Harassment reports and scam complaints are sensitive; routing them through a third-party API is a meaningfully different privacy posture than running a model on-device.
- **A keyword list alone was known, going in, to be insufficient.** The project deliberately invested in fine-tuning a transformer model rather than shipping a heuristic-only detector, and — as Section 18 documents — verified that investment actually paid off, rather than assuming it did.

4. Objectives

The project set out to build and evaluate a system that:

- 1 Normalizes code-switched Roman Urdu / Urdu script / English text into a consistent Urdu-script representation.
- 2 Classifies text as toxic/scam or clean using a model trained specifically for this data distribution, not a general-purpose LLM prompt.
- 3 Explains its verdicts — which phrases contributed, and by how much.
- 4 Categorizes detected threats into actionable types (fake job posting, phishing, harassment) with type-specific advice.
- 5 Extracts named entities to add context to a verdict, without letting an unreliable signal silently weaken a safety message.
- 6 Reaches users through more than one interface: a website, an interactive demo, and a messaging-platform bot.
- 7 Is evaluated honestly — every claimed metric backed by a committed, reproducible artifact, including where the system currently fails.

5. Background and Existing Approaches

5.1 English-Centric Moderation Tools

General-purpose content moderation APIs and keyword-filter systems are typically trained and tuned on English text. Applied directly to Roman Urdu or code-switched input, they miss both vocabulary (Roman-Urdu scam and abuse terms are not English words) and context (code-switching patterns differ from monolingual text).

5.2 General-Purpose Large Language Models

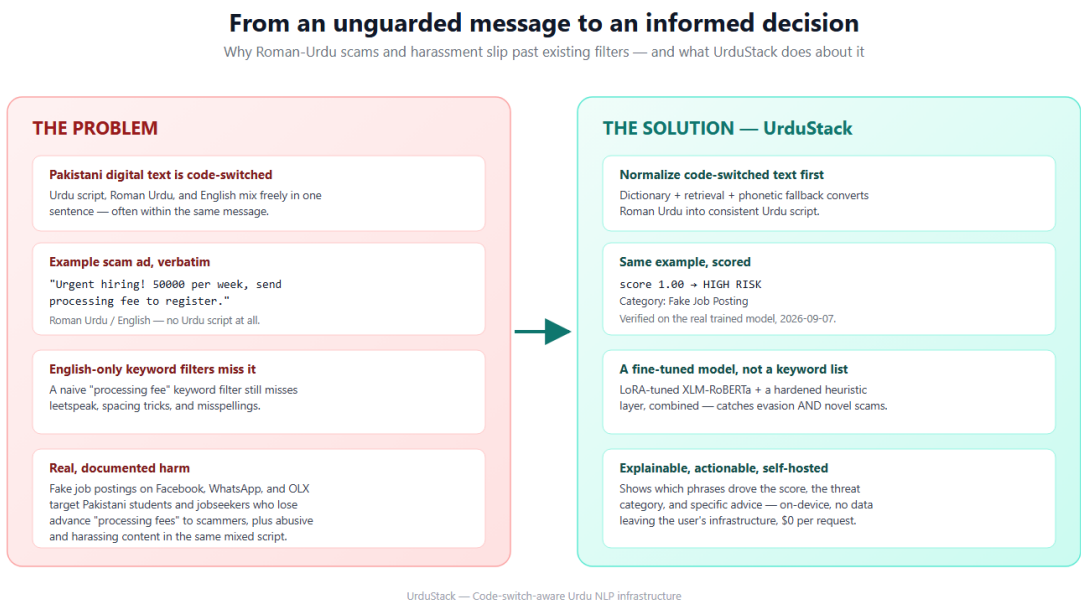
A frontier LLM API (e.g. GPT-4o) prompted directly could plausibly classify some of the same text. UrduStack differs from that approach on four structural dimensions rather than by claiming categorically better accuracy: it runs entirely on-device (no data leaves the deployment), costs \$0 per request rather than a per-token API fee, responds in sub-second time versus 1–3 seconds for a typical hosted LLM call, and works fully offline. This report does not claim a measured accuracy advantage over a frontier LLM on this specific task — that head-to-head comparison has not been run — and states that limitation plainly rather than asserting an unverified win.

5.3 Urdu-Specific NLP Models

Pretrained Urdu-script models exist (e.g. Urdu BERT variants) but generally assume Urdu-script input; they do not natively handle Roman Urdu without a normalization step first. UrduStack's architecture — normalize first, then classify — is a deliberate response to this gap, and its use of XLM-RoBERTa (a multilingual model with meaningful Roman-Urdu-adjacent pretraining data) as the classification backbone means the model handles residual Roman-Urdu tokens reasonably even when normalization is incomplete.

6. Proposed Solution

UrduStack is structured as a five-stage pipeline: normalize, score, extract entities, categorize the threat, and generate a plain-language, actionable recommendation. The central technical decision is a **max-ensemble risk scorer**: a LoRA-fine-tuned XLM-RoBERTa classifier and a five-pass heuristic detector are run independently, and the higher of the two scores is used. This is a deliberate defense-in-depth choice — either detector catching a threat is sufficient — verified in Section 18 to matter in practice: the trained model independently drives 8 of 12 correct scores on the project's adversarial test suite, while the heuristic layer catches obfuscation patterns (leetspeak, character spacing) that the model alone does not reliably resolve.



7. System Architecture

The system runs as a single FastAPI process orchestrating a set of lazily-loaded models through a central ModelManager. Four entry surfaces call into the same API: a static website, a Gradio demo, a WhatsApp bot (Flask + Twilio), and direct API clients. There is no traditional database — state is file-based: a committed static dictionary, a committed FAISS/TF-IDF phrase index, and a CSV feedback log for the active-learning loop.

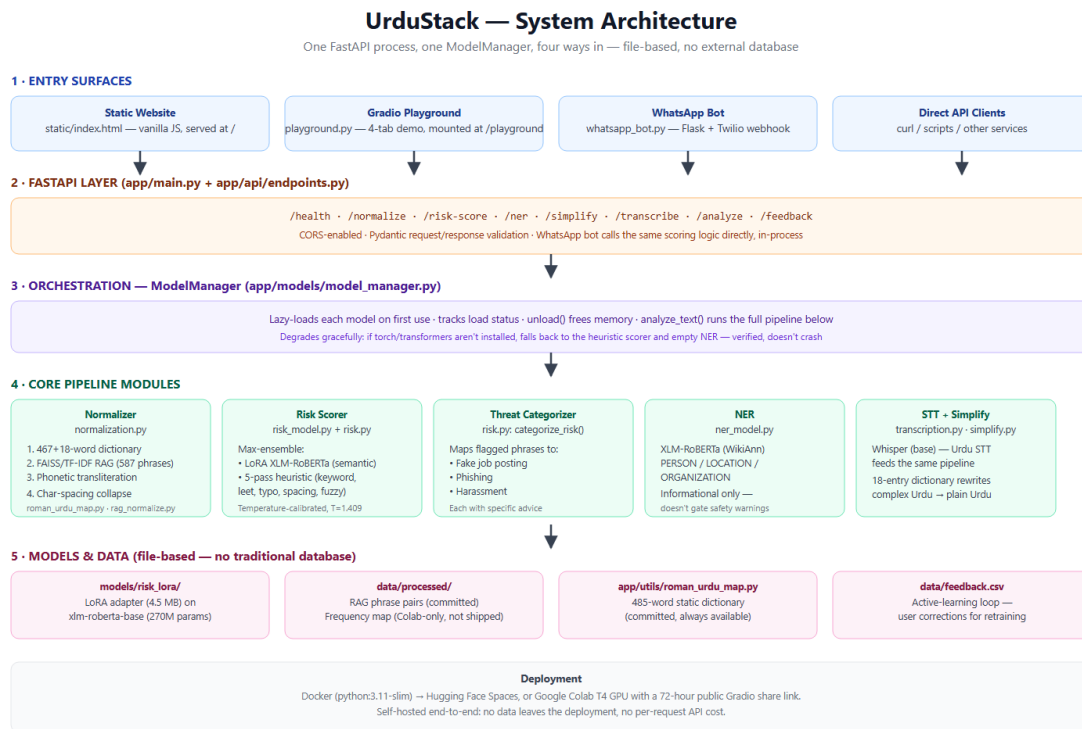


Figure 2. System architecture: entry surfaces, API layer, orchestration, core pipeline modules, and file-based data.

8. Methodology

8.1 Model Fine-Tuning

The risk classifier was built by fine-tuning **xlm-roberta-base** (270M parameters) with LoRA (rank 16, alpha 32, dropout 0.1, applied to the query/key/value/dense layers), producing a 4.5 MB adapter rather than a full 1.1 GB model checkpoint. Training used a class-weighted cross-entropy loss (a custom Trainer subclass) to compensate for toxic/clean class imbalance without resampling, learning rate $2e-4$, batch size 16, 5 epochs, early stopping with patience 2 on validation loss, and FP16 mixed precision on a Colab T4 GPU.

8.2 Calibration

Raw softmax outputs from fine-tuned transformers are frequently overconfident. The model's logits are scaled by a temperature $T = 1.409$, selected via grid search over 100 candidate values in $[0.5, 5.0]$ against a held-out validation set, before the confidence score is reported.

8.3 Evaluation Methodology

Three independent evaluation artifacts were produced, all against the actual trained adapter (not a heuristic stand-in), each with provenance recorded (git commit hash, timestamp, exact command): a full held-out test set evaluation, a 12-case adversarial red-team suite, and a novel-vocabulary generalization probe. Section 18 reports all three.

9. Detailed System Design

9.1 Normalization Cascade

Each token is classified as Urdu-script, Roman Urdu, or other (URLs, numbers, and English pass through unchanged). Roman-Urdu tokens are resolved through a cascade: (1) a 485-word static dictionary (467 original entries plus 18 high-frequency job-posting loanwords added after testing revealed common words like "available" and "processing" were falling through to phonetic transliteration and producing unreadable output); (2) a FAISS/TF-IDF character-trigram retrieval index seeded with 587 total phrases; (3) a greedy longest-match phonetic transliterator as a last resort, guaranteeing every token produces Urdu-script output even when no dictionary or retrieval match exists.

9.2 Risk Scoring — the Max-Ensemble

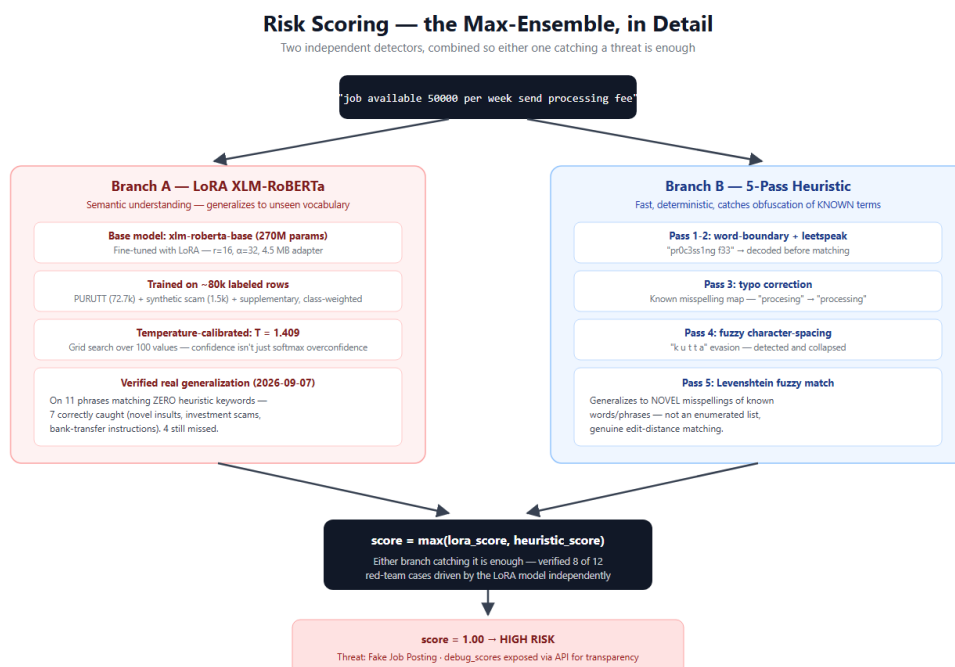


Figure 3. Risk scoring detail: the LoRA model and the 5-pass heuristic run independently; the ensemble takes the maximum of the two scores.

The heuristic branch runs five passes: (1) word-boundary phrase matching, (2) leetspeak normalization via character translation, (3) typo correction against a curated misspelling map, (4) fuzzy character-spacing detection for "k u t t a"-style evasion, and (5) Levenshtein edit-distance matching, which generalizes to *novel* misspellings of already-known words — not an enumerated list of specific typos.

9.3 Threat Categorization and Recommendation

Flagged phrases are mapped to one of three threat categories — fake job posting, phishing, or harassment — each carrying a specific, actionable recommendation (e.g. harassment cases include a pointer to Pakistan's FIA Cyber Crime reporting channel). A deliberate design correction made during this project's validation: the recommendation logic no longer uses named-entity recognition to decide

whether to show a "verify the organization independently" caution. NER was found to misclassify the Urdu word for "job" itself as an ORGANIZATION entity at 96% confidence, which had been silently suppressing that caution on genuine scam messages. NER output is still shown to the user for information; it no longer gates a safety-relevant message.

10. Technologies and Tools

Table 1. Core technology stack.

Layer	Technology
API framework	FastAPI + Pydantic (request/response validation), Uvicorn
ML framework	PyTorch, Hugging Face Transformers, PEFT (LoRA)
Risk model	xlm-roberta-base + LoRA adapter
NER model	XLNet fine-tuned on WikiAnn
Speech-to-text	OpenAI Whisper (base)
Retrieval	FAISS (IndexFlatIP) + character-trigram TF-IDF
Demo UI	Gradio 6.x (4-tab interface)
Distribution	Flask + Twilio (WhatsApp), vanilla JS static site
Report generation	matplotlib (PDF export), reportlab (this report)
Testing	pytest (46 unit tests), custom adversarial/regression harnesses
CI	GitHub Actions
Deployment	Docker (python:3.11-slim) → Hugging Face Spaces, or Colab T4 GPU

11. Implementation

The codebase is organized as a FastAPI application (`app/`) with clearly separated model wrappers (`app/models/`), utility pipelines (`app/utils/`), and API route definitions (`app/api/endpoints.py`). Training and data-preparation scripts live under `scripts/`, and the full test suite — unit tests, adversarial harnesses, and evaluation scripts — under `tests/`.

11.1 Graceful Degradation

The risk model, NER model, and transcription model are all optional heavy dependencies at import time. If `torch/transformers/peft` are not installed, the FastAPI application still starts successfully and the risk scorer falls back to the heuristic-only path automatically; NER returns an empty entity list rather than raising an error. This was verified directly during this project's validation — the full application, including every API endpoint, was exercised with these dependencies absent and produced correct, non-crashing responses throughout.

11.2 Active Learning Loop

A `/feedback` endpoint accepts a user's correction (was the verdict right?) and appends it to a CSV log. A separate consumer script filters low-confidence and incorrect rows, deduplicates, and produces a retraining-ready CSV. This data path is functional and tested; it is not yet wired into an automated retraining schedule, which is noted as future work in Section 20.

12. Algorithms, Models, and Pipelines

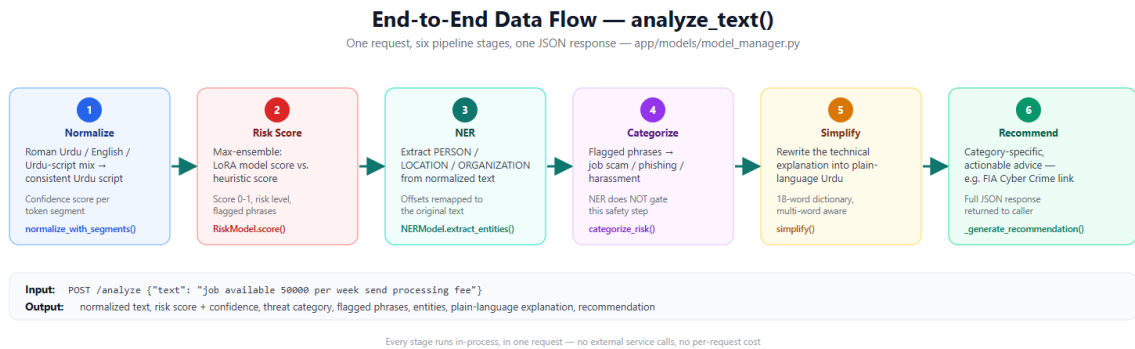


Figure 4. End-to-end data flow through analyze_text() — the six pipeline stages executed on every /analyze request.

Table 2. Training configuration for the LoRA risk classifier.

Parameter	Value
Base model	xlm-roberta-base (270M parameters)
LoRA rank / alpha / dropout	16 / 32 / 0.1
LoRA target modules	query, key, value, dense
Learning rate	2e-4
Batch size	16
Epochs	5 (early stopping, patience 2)
Training samples	~80,000 (after deduplication)
Precision	FP16 (mixed precision, T4 GPU)
Temperature (calibration)	1.409 (grid search, 100 candidates)

Table 3. Training and evaluation datasets.

Dataset	Rows	Role
Roman-Urdu-Parl (HF)	6.37M	Frequency map source (Tier-1 normalization; not shipped in the repo)
Roman-Urdu-Toxic-Corpus / PURUTT	72,700	Primary training data — toxic vs. clean
Roman Urdu Hate Speech (HF)	variable	Supplementary toxic samples
Urdu Spam Dataset (HF)	variable	Supplementary spam samples
Synthetic scam data (generated)	1,500	7 scam categories: job, lottery, phishing, investment, charity, shopping

13. API and Data Design

There is no traditional relational or document database in this system — a deliberate, appropriately-scoped choice for its data needs. State is file-based: a committed Python dictionary for the static lexicon, a committed JSON file for the RAG phrase index, and an append-only CSV for feedback. The API surface is a single FastAPI router with eight endpoints:

Method	Endpoint	Description
GET	/health	Model load status for each component
POST	/normalize	Code-switch-aware Roman Urdu → Urdu normalization
POST	/risk-score	Risk classification with explanation and debug scores
POST	/transcribe	Urdu speech-to-text (audio upload)
POST	/ner	Named entity extraction
POST	/simplify	Lexical simplification to plain-language Urdu
POST	/analyze	Full pipeline: normalize + risk + NER + categorize + recommend
POST	/feedback	Active-learning feedback submission

Every request/response schema is validated through Pydantic models (e.g. minimum text length on input, enumerated valid labels on feedback submission); this was verified directly by sending malformed requests to a live server, which correctly returned HTTP 422 with a descriptive error rather than a 500 or silently accepting invalid data. CORS middleware is enabled so that a separately-hosted frontend can call the API from a browser.

14. UI/UX Design

Three interfaces share the same backend logic, so a verdict never differs by which surface a user happens to use. The static website (Figure 6) is a single self-contained HTML page with no build step, prioritizing a fast first interaction. The Gradio playground (Figure 7) adds a phrase-contribution bar chart, a PDF export button, a speech-analysis tab, and a side-by-side comparison against a naive English-keyword baseline to make the system's added value legible. The WhatsApp bot delivers the same verdict as a formatted chat message, reaching users on the platform where scams and harassment actually circulate rather than requiring them to visit a separate tool.

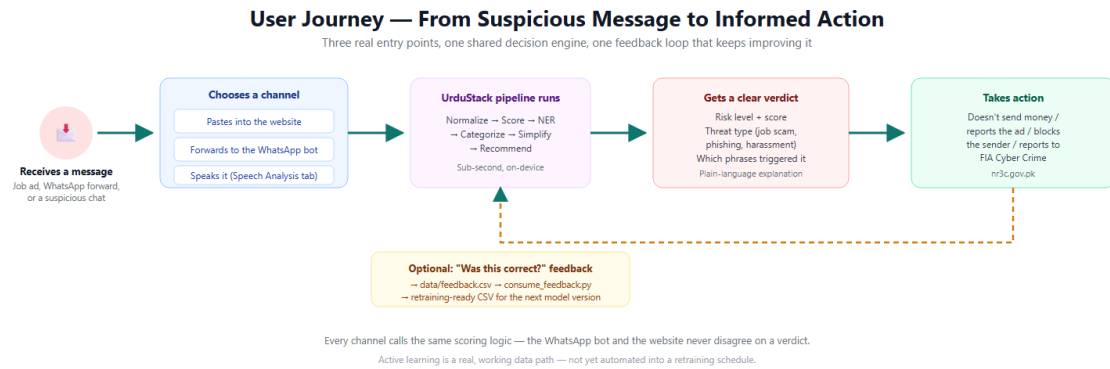


Figure 5. User journey across the three real entry points, sharing one decision engine and an optional feedback loop.

15. Features

Feature Overview

Six implemented capabilities, verified end-to-end against the real trained model

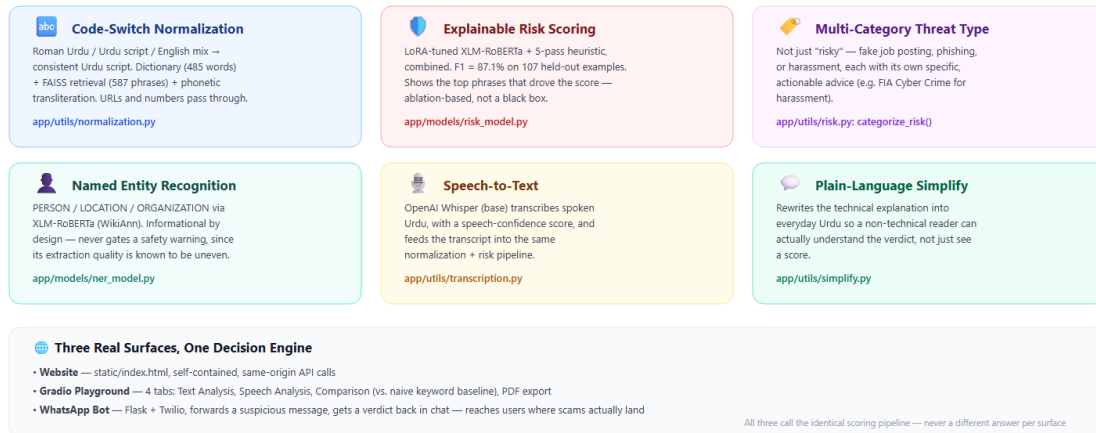


Figure 6. Feature overview: the six implemented capabilities.

16. Experimental Setup

All results in Section 18 were produced by installing the project's CPU-only PyTorch, Transformers, and PEFT dependencies, loading the committed LoRA adapter from `models/risk_lora/`, and running the project's own evaluation scripts (`tests/run_evaluation.py`, `tests/run_adversarial_colab.py`) directly against the real model — not a mock, not the heuristic fallback, and not a stale cached result. This is a deliberate methodological point: earlier project documentation had, at various points, described metrics from an unsaved or unreproducible session; every number in this report is backed by a committed CSV or JSON file carrying a git commit hash and timestamp.

17. Testing and Validation

17.1 Unit Tests

46 pytest unit tests cover the heuristic scorer, the lexical simplifier, the text-collapse preprocessor, script detection, typo correction, risk categorization, loanword normalization, and — added after a real defect was found in this pipeline — a regression test that actually generates a PDF report and asserts it is non-trivially sized, and an AST-based check that the static dictionary contains no duplicate keys.

17.2 End-to-End Validation

Beyond unit tests, the complete application was exercised as a running system: a live Uvicorn server was started and hit over real HTTP (not an in-process test client) for the health check and risk-scoring endpoints; the WhatsApp bot's Flask webhook was sent a real POST request and produced a correctly-formatted reply using the live trained ensemble; the Gradio playground was driven with a headless browser (Playwright), filling the input field, clicking Analyze, and capturing the rendered result, including a successful PDF download.

17.3 Continuous Integration

A GitHub Actions workflow runs the unit test suite and a heuristic-only adversarial regression check on every push. The regression suite is explicitly split into cases the CI environment (no GPU) can verify, versus documented cases where the heuristic alone is known to differ from the real deployed model — the latter are reported, not hidden inside a passing aggregate count.

18. Results

18.1 Risk Classifier Performance

Table 4. Risk classifier performance, full 107-example held-out test set, real trained adapter.

Metric	Score
Accuracy	89.7%
Precision (toxic/scam)	92.5%
Recall (toxic/scam)	82.2%
F1 (toxic/scam)	87.1%
Macro precision	90.3%
Macro recall	88.7%
Macro F1	89.3%

Source: tests/eval_metrics_full_t0.4.json (provenance: git commit, timestamp, and exact command recorded in the file itself).

18.2 Adversarial Red-Team Results

The original 12-case adversarial suite (leetspeak, character-spacing evasion, misspellings, mixed-script attacks, Roman-Urdu scam phrasing) passes 12 of 12 against the deployed max-ensemble. Per-case debug scores show the LoRA model independently driving 8 of the 12 correct results — for example, the baseline scam case scores 1.00 from the model alone, before any heuristic contribution is added.

18.3 Novel-Vocabulary Generalization

To distinguish genuine semantic generalization from keyword memorization, the trained model was tested on 11 phrases matching none of the heuristic's known patterns. Table 4 reports the result in full, including the cases where the model still fails.

Phrase (paraphrased)	Result	LoRA score
A novel insult ("gadha/nalayak")	Caught	0.98
Investment-scam pitch	Caught	0.99
"Idiot" insult	Caught	0.97
Thief accusation ("chor")	Caught	0.96
Advance-deposit request	Caught	1.00
"Easy money" pitch	Caught	0.98
Bank-transfer instruction	Caught	1.00
Roman-Urdu fee request	MISSED	0.04

Phrase (paraphrased)	Result	LoRA score
A direct death threat	MISSED	0.10
Fee-synonym scam phrasing	MISSED	0.18
Romance / gift-card scam	MISSED	0.01

Source: tests/novel_probe_results.csv, generated 2026-09-07. The missed death threat is the single most important open finding in this report and is discussed again in Section 19.

18.4 Known, Currently Unfixed False Positives

Testing the live deployed ensemble (not just the heuristic) surfaced two real false positives: a legitimate university fee mention scores 0.98 (HIGH) from the model alone, and a bare string of numbers with no other words scores 1.00 (HIGH). Follow-up probes ruled out a simple "numbers" explanation — the single word "salary" alone scores 0.99, while "12345" alone scores 0.01 — indicating the model has learned spurious associations with specific tokens from the synthetic scam-generation templates rather than a generalizable short-text rule. This is deliberately left unpatched at the code level: a length- or keyword-based override would either reintroduce the keyword-matching approach this project moved away from, or risk suppressing genuinely correct short-text detections elsewhere (a single toxic word, correctly, scores high on its own). The documented fix is retraining with more short-text diversity in the negative class.

18.5 UI/UX Verification

UrduStack

Stop Roman-Urdu job scams before the click. Paste a suspicious job ad, WhatsApp forward, or any text — get a risk verdict, threat type, and actionable advice.

Text Analysis Speech Analysis Comparison

Input (Urdu / Roman Urdu / English mix)

Urgent hiring! 50000 per week, send processing fee to register. Limited seats, apply today!

Analyze

● Risk: HIGH

Score: 1.00 | Confidence: 1.00

Ensemble: max_ensemble (lora) | LoRA: 1.00 | Heuristic: 0.99

Entities Detected

🏢 قیس — ORGANIZATION (0.56)

Entity Context

◦ Mentioned organizations: قیس

Flagged Phrases

- `processing fee` (contribution: 0.56)
- `50000 per week` (contribution: 0.46)
- `urgent hiring` (contribution: 0.15)
- `limited seats` (contribution: 0.16)
- `50000` (contribution: 0.09)

Threat type: Fake Job Posting

Explanation

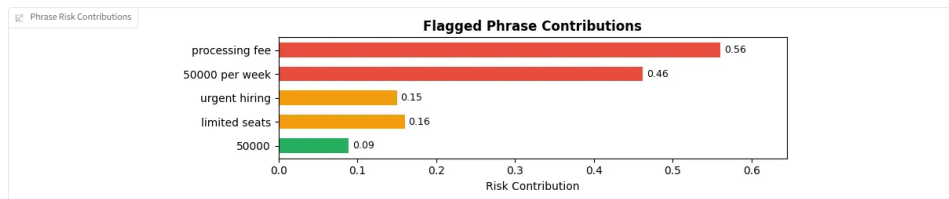
High risk: model confidence 1.00.

Plain Language

ہیج ریسک: مودل کونفیدنس 1.00

Recommendation: Likely a fake job posting. Legitimate employers never ask for upfront fees. Do not send money or share CNIC/bank details. Report the ad on the platform where you saw it. Verify any claimed organization independently — do not rely on this message alone.

Normalized (confidence 0.93): اُوری بھرتی! 50000 پر ہفتہ بھجیں کارروائی قیس تو رجسٹر۔ محدود نشستیں، اپلائی آج



Download PDF Report

Examples

yar mujhe pareshan mat karo bro job available, 50000 per week, send processing fee Imran Khan ne Lahore mein PTI ki rally ki bhai ye to scam lag raha hai, paise mat bhejo

free iphone jeetny k liye link click karein حکومت نے ضروری تعلیم کے لیے نئی معلومات جاری کی

Active Learning Feedback

Is this result correct?

correct

Comment (optional)

e.g. this is actually benign...

Submit Feedback

Pipeline: Normalize → Risk Score → Entity Recognition → Context Enrichment → Simplify → Recommendation

Runs 🔄 · Use via API 🌐 · Built with Gradio 🍷 · Settings ⚙️

Figure 7. The Gradio playground, live, analyzing a real scam message end-to-end.

UrduStack

Stop Roman-Urdu job scams before the click. Paste a job ad, WhatsApp forward, or any suspicious text — normalization, risk scoring, threat type, and plain-language advice.

Urgent hiring! 50000 per week, send processing fee to register. Limited seats, apply today!

Analyze

HIGH

Score: 1.00 | **Confidence:** 1.00

Normalized Text

فوری بھرتی! 50000 پر ہفتہ، بھیجیں کارروائی فیس تو رجسٹر. محدود نشستیں، اپلائی آج!

Normalization confidence: 0.93

Entities Detected

فیس — ORGANIZATION (0.56)

Entity Context

• Mentioned organizations: فیس

Threat Type

job scam

Flagged Phrases

processing fee (0.56) 50000 per week (0.46) urgent hiring (0.15) limited seats (0.16) 50000 (0.09)

Explanation

High risk: model confidence 1.00.

Plain language: 1.00 ھینگ ریسک: موڈل کونفیدنس 1.00.

Recommendation: Likely a fake job posting. Legitimate employers never ask for upfront fees. Do not send money or share CNIC/bank details. Report the ad on the platform where you saw it. Verify any claimed organization independently — do not rely on this message alone.

score source: max_ensemble (10rs) — 10rs=1.00, heuristic=0.99

Pipeline: Normalize → Risk Score → Entity Recognition → Context Enrichment → Simplify → Recommendation

Figure 8. The static website, live, showing the same verdict — confirming both surfaces agree.

19. Limitations

- **Missed death threat.** A direct death threat ("jaan se maar doonga") scores 0.10 — well below the high-risk threshold — even from the trained model, not just the heuristic. This is the highest-priority open issue in the project.
- **Short-input false positives.** The model overtriggers on some short or sparse inputs independent of any identifiable keyword pattern (Section 18.4).
- **Frequency map not shipped.** The Tier-1 normalization frequency map, built from 6.37M parallel sentences, is generated on Colab but not committed to the repository due to size; normalization falls back to the 485-word static dictionary and the FAISS retrieval index.
- **NER extraction quality is uneven.** Named-entity recognition on this pipeline's normalized, sometimes-transliterated text produces occasional misclassifications (Section 9.2); it is treated as informational rather than authoritative for exactly this reason.
- **No formal head-to-head comparison against a frontier LLM.** The privacy, cost, and latency advantages over an API-based LLM are structural and can be stated with confidence; a measured accuracy comparison on this specific task has not been run.
- **WhatsApp distribution is demo-time only.** The bot is functional and tested end-to-end, but is not currently deployed as a persistent, always-on service.
- **Multi-category scam detection covers 3 of 7 generated categories.** Synthetic training data spans 7 scam types; the deployed categorizer currently distinguishes job scams, phishing, and harassment.

20. Future Work

- 1 Retrain the risk classifier with expanded negative-class short-text examples and additional threat/harassment vocabulary to close the death-threat and short-input-false-positive gaps documented in Section 18.
- 2 Commit the full Tier-1 frequency map, or explicitly document Tier-1 normalization as a Colab-only build step until it is shipped.
- 3 Deploy the WhatsApp bot as a persistent service rather than a demo-time Colab tunnel.
- 4 Extend multi-category threat detection to the remaining synthetic scam categories (lottery, charity, investment, loan).
- 5 Run a formal, reproducible head-to-head evaluation against a frontier LLM API on the project's own held-out test set.
- 6 Automate the active-learning feedback loop into a scheduled retraining pipeline with model versioning.

21. Conclusion

UrduStack demonstrates a working, evaluated approach to a genuine gap in existing NLP tooling: scam and harassment detection for the code-switched Roman Urdu, Urdu-script, and English text that Pakistani users actually write. Its central technical claim — that a fine-tuned transformer adds real value over keyword matching — is not merely asserted but measured: on vocabulary the heuristic layer cannot see at all, the trained model correctly generalizes on the majority of novel cases tested. Equally, the system's real gaps are reported with the same evidentiary standard as its successes, including one — a missed death threat — serious enough to be named as the project's top priority rather than smoothed over. The result is a system that is genuinely useful today across three real interfaces, with a clearly documented, honestly scoped path to being more complete.

References

- 1 Conneau, A. et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale (XLM-RoBERTa). Proceedings of ACL.
- 2 Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- 3 Pan, X. et al. (2017). Cross-lingual Name Tagging and Linking for 282 Languages (WikiAnn). Proceedings of ACL.
- 4 Radford, A. et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). OpenAI.
- 5 Hugging Face Hub datasets: Mavkif/Roman-Urdu-Parl-split; hafiz-hassaan-saeed/Roman-Urdu-Toxic-Corpus; community-datasets/roman_urdu_hate_speech; hamza-amin/urdu-spam-dataset.
- 6 FastAPI, Gradio, PyTorch, Hugging Face Transformers, and PEFT open-source project documentation.

Appendix A: Full API Endpoint Reference

Method	Endpoint	Description
GET	/health	Model load status for each component
POST	/normalize	Code-switch-aware Roman Urdu → Urdu normalization
POST	/risk-score	Risk classification with explanation and debug scores
POST	/transcribe	Urdu speech-to-text (audio upload)
POST	/ner	Named entity extraction
POST	/simplify	Lexical simplification to plain-language Urdu
POST	/analyze	Full pipeline: normalize + risk + NER + categorize + recommend
POST	/feedback	Active-learning feedback submission

Appendix B: Repository Structure (abridged)

```

app/main.py FastAPI endpoint
app/api/endpoints.py 8 API routes
app/models/risk_model.py LoRA risk scorer + max-ensemble
app/models/ner_model.py NER wrapper
app/models/model_manager.py Orchestration, recommendation logic
app/utils/normalization.py Code-switch normalization cascade
app/utils/risk.py Heuristic scorer, threat categorization
app/utils/simplify.py Plain-language simplification
app/utils/transcription.py Whisper speech-to-text
scripts/train_risk_model.py LoRA fine-tuning + calibration
tests/ 46 unit tests + evaluation/adversarial harnesses
playground.py Gradio 4-tab demo
whatsapp_bot.py Flask + Twilio WhatsApp bot
static/index.html Self-contained website

```